UNIVERSITÀ DEGLI STUDI DI SALERNO

SMAAF - Static Malware Automation Analysis Framework

A modular framework for static malware automation analysis.

Prof. A. Castiglione

Antonio Garofalo

Presentation Schedule

01

Phase 1

Goals and Motivations

02

Phase 2

Framework Flow and Technologies

03

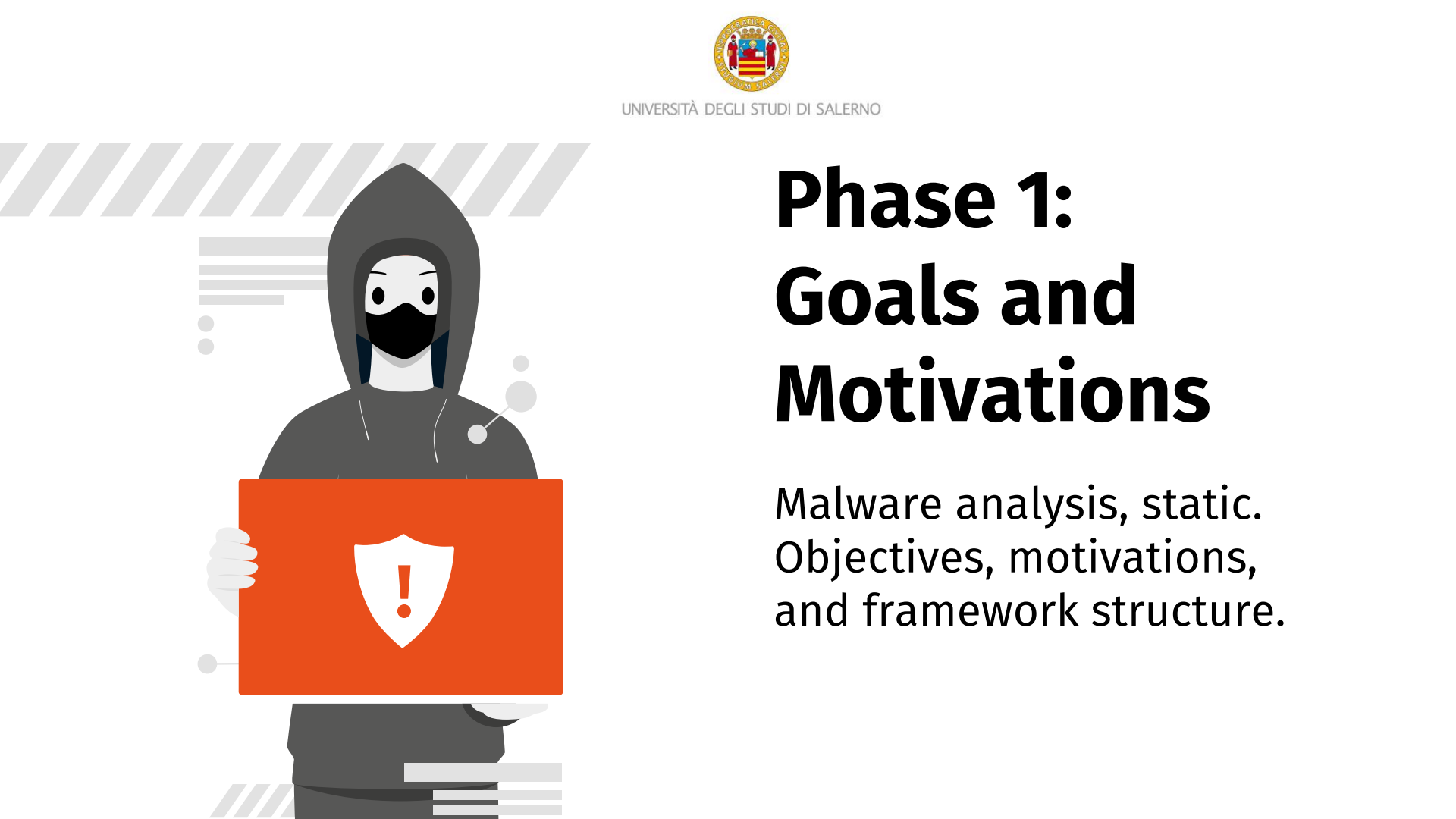
Phase 3

Analysis of SMAAF Modules

04

Phase 4

Conclusions and Future Devs



Phase 1: Goals and Motivations

Malware analysis, static. Objectives, motivations, and framework structure.

Malwares Types Overview

Credentials Stealers

Exfiltration of credentials and sensitive data.

Worms

It spreads through a network by replicating itself.

Botnet/ RAT

Remote control, C2 orchestration, DDoS.

Ransomware

Malware that, when executed, initiates data encryption.

Malware Stats Overview

>450K/day

New malware and PUA samples recorded daily (AV-TEST estimate).

27%

Malware was involved in ~27% of breaches in the public sector (Verizon DBIR 2024).

~28%

Percentage of ransomware-related cases among incidents analyzed (IBM X-Force 2025)

61%

In malware-related breaches, ransomware was the dominant type in many sectors (e.g., 61% of malware-related cases).

Static Malware Analysis

First Responder

Need for rapid response in critical incidents.

Modules

Exclude exclusive sample analysis and reuse the code.

Risk Reduction

Safe analysis without execution.

STIX/TAXII

Enable automation and intelligence sharing (STIX/TAXII).

SMAAF Framework Modules

Malware Sample Collector

Gathers samples from honeypots, malware feeds, submissions.

String and Signature Extractor

Extracts Artefacts that are useful for the Behavioral Predictor.

Reporting Engine

Summarizes the workflow and use it for a Threat Intelligence process.

Uses disassemblers like IDA Pro, Radare2, Ghidra and translate bin to assembly.

Disassembler Module

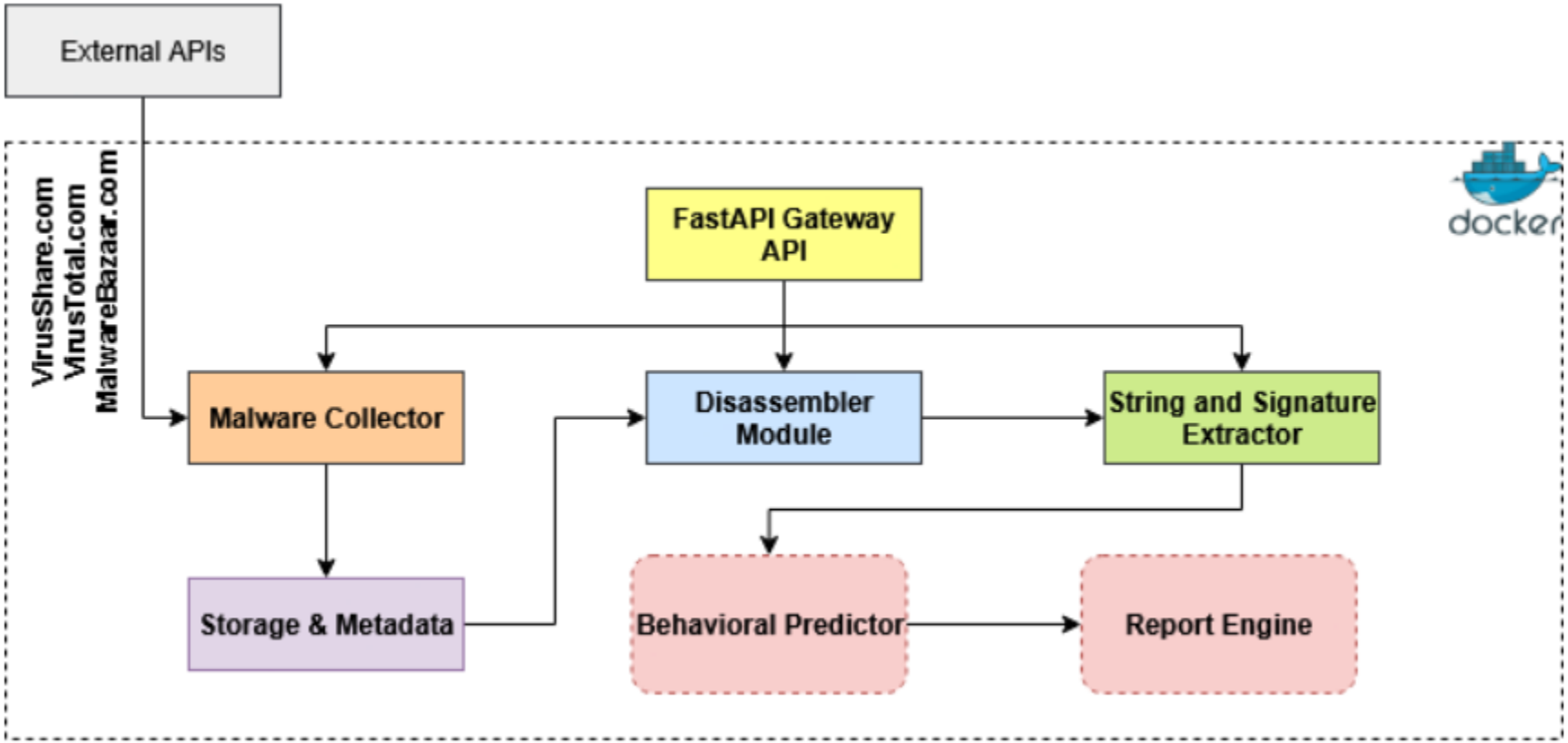
ML model that predicts malware capabilities statically.

Behavioral Predictor

Phase 2: Framework Flow and Technologies

Analysis of the Flow Framework and an overview of the technologies used for development.

Framework Flow



Technologies

(malware sample collector)

FastAPI

Lightweight and fast Python framework for exposing APIs (upload, job trigger, status) and orchestrating the pipeline.

Docker

Containerization to isolate components, ensure reproducibility, and simplify deployment.

APIs

Integration of feeds and APIs to automatically find, verify, and enrich samples (reputation, metadata, and files) to support the catalog and triage.

Technologies

(disassembler module)

Ghidra

Ghidra is an open-source disassembler that runs in headless mode, allowing batch analysis of binaries and automatic extraction of assembly and pseudocode through scripting.

radare2

Radare2 is a highly scriptable toolkit for binary analysis that speeds up the extraction of sections, functions, and metadata and integrates easily into Python pipelines.

Technologies

(string and signature extractor)

YARA Rules

YARA allows you to define rules based on strings and patterns to identify malware families and known signatures, enabling automatic escalation of samples that match the rules.

Strings

strings floss and rabin2 allow you to quickly extract readable strings from binaries and, with floss, to deobfuscate hidden strings to reveal URLs, IPs, and other indicators useful for extracting IOCs.

Phase 3: Analysis of SMAAF Modules

Detailed explanation of the modules implemented with analysis of the results.

Malware Sample Collector

VirusTotal

Provides multi-engine malware scanning and signature matching to verify suspicious files.

VirusShare

Offers a large repository of real-world malware samples for research and analysis.

Malware Bazaar

Distributes verified malware samples with metadata and threat intelligence tags.

User Upload

A simple User Interaction Portal.

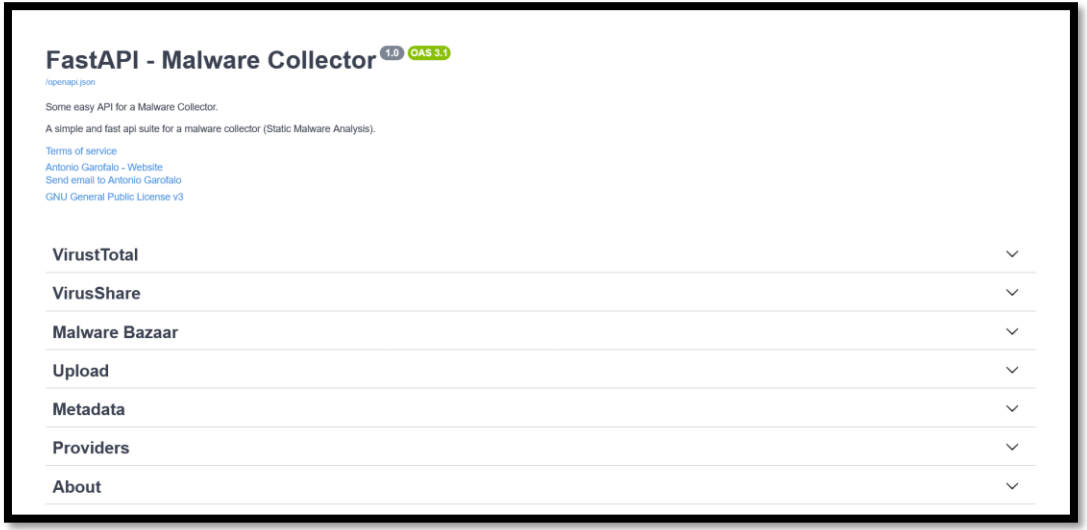
FastAPI Server

The **FastAPI** server provides a RESTful interface that allows interaction between the internal modules of the framework.

What can do FastAPI for the SMAAF?

It exposes dedicated endpoints for collecting malware samples, retrieving metadata, and communicating with external portals (**VirusTotal, VirusShare, MalwareBazaar**).

The automatically generated **Swagger UI** simplifies testing and documentation of all available APIs, ensuring modular, secure, and isolated communication.

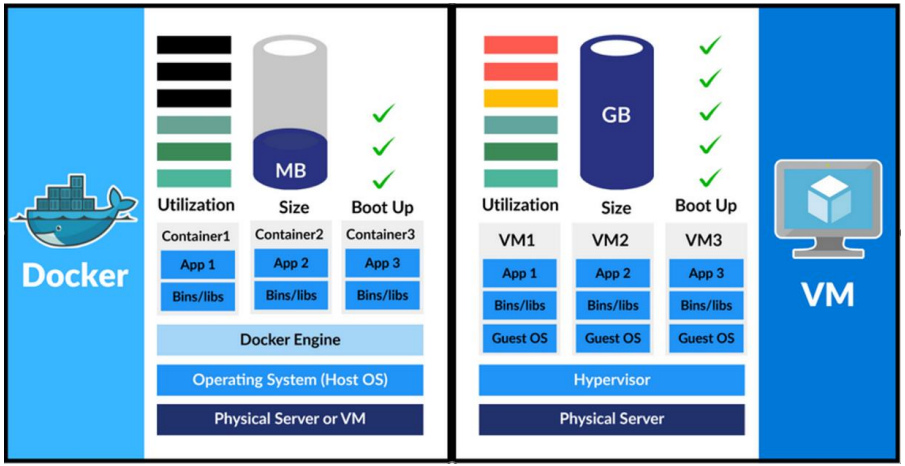


Isolated Environments

The framework uses isolated environments to ensure secure handling of malware samples.

Containers

Containers (Docker) provide lightweight and fast deployment for automation tasks, enabling multiple collectors to run independently with shared host resources. They are ideal for scalable and modular API-based operations.



Virtual Environments

Virtual Environments (VMs) offer deeper isolation with dedicated operating systems, making them suitable for analyzing high-risk samples or performing more resource-intensive tasks.

Ghidra

Open-source disassembler & decompiler with powerful headless automation.

What is Ghidra?

- Converts binary executables (PE, ELF, APK) into **assembly and C-like pseudocode**.
- Performs **architecture detection** (x86, x64, ARM, MIPS, etc.) automatically.
- Supports **headless batch processing** for automated workflows.
- Allows **Python/Java scripting** for custom export and feature extraction.

The Ghidra Project

radare2

Command-line framework for fast, scriptable binary analysis and feature extraction.

What is radare2?

- Extracts **.text / .data sections**, **symbols**, and **function tables**.
- Opens and analyzes **binaries in headless mode** for automation.
- Detects **imports, exports, and strings (ASCII/Unicode)**.
- Identifies **architecture and entry points** from headers and disassembly.
- Ideal for **scalable and parallel sample processing**.

The radare2 Project

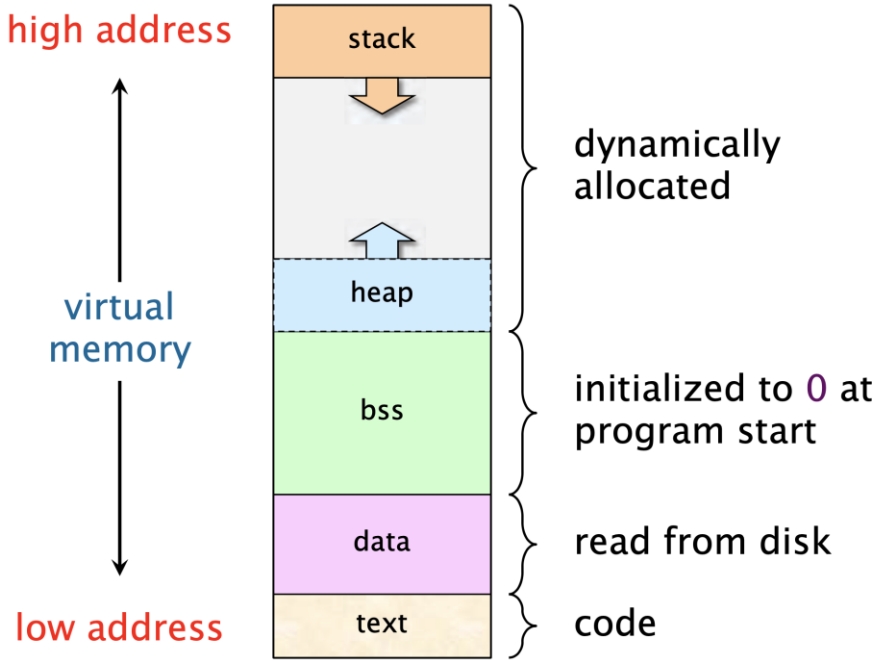
Recovered Data

What we need?

During disassembly, the module extracts and reconstructs memory sections from the binary file.

- **.text** → executable code recovered for instruction flow analysis.
- **.data** → initialized global/static variables read from disk.
- **.bss** → uninitialized data, allocated at program start.
- **heap / stack** → dynamically allocated regions identified for behavior prediction.

These recovered sections allow the framework to map code, data, and memory references for deeper static analysis.



String and Signature Extractor

The String and Signature Extractor module identifies readable and obfuscated strings from the disassembled binary.

“FLOSS”

FLOSS (FireEye Labs): automatically detects and decodes obfuscated or dynamically constructed strings used by malware.

Strings (or rabin2): extracts plain ASCII and Unicode text embedded in the binary (URLs, IPs, registry keys, function names).

The extracted strings are analyzed for **Indicators of Compromise (IOCs)** and matched against **YARA rules** to detect known malware families.

YARA Rules

What is YARA Project?

The **YARA** module is used to identify malware families and known behavioral patterns through static signature matching.

•Each **YARA rule** defines a set of **string patterns, byte sequences, or code fragments** associated with a specific threat.

•The framework integrates **custom rules** and **public feeds** (e.g., VirusTotal YARA repository).

•When a rule matches, the malware sample is tagged with the corresponding **family or signature name**.

In this example, the rule detects a **Stuxnet sample**, one of the most well-known industrial malware cases.

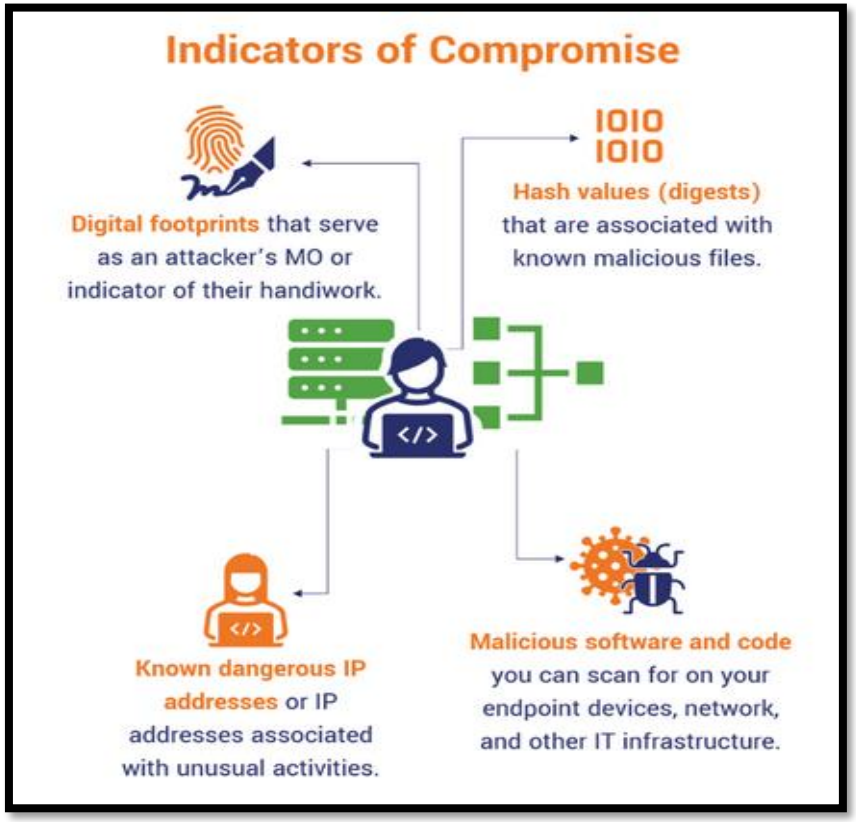
```
meta:
    description = "Stuxnet Sample - file malware.exe"
    author = "Florian Roth"
    reference = "Internal Research"
    date = "2016-07-09"
    hash1 = "9c891edb5da763398969b6aaa86a5d46971bd28a455b20c2067cb512c9f9a0f8"

strings:
    // 0x10001778 8b 45 08  mov     eax, dword ptr [ebp + 8]
    // 0x1000177b 35 dd 79 19 ae    xor     eax, 0xae1979dd
    // 0x10001780 33 c9     xor     ecx, ecx
    // 0x10001782 8b 55 08  mov     edx, dword ptr [ebp + 8]
    // 0x10001785 89 02     mov     dword ptr [edx], eax
    // 0x10001787 89 ?? ??  mov     dword ptr [edx + 4], ecx
    $op1 = { 8b 45 08 35 dd 79 19 ae 33 c9 8b 55 08 89 02 89 }
    // 0x10002045 74 36     je      0x1000207d
    // 0x10002047 8b 7f 08  mov     edi, dword ptr [edi + 8]
    // 0x1000204a 83 ff 00  cmp     edi, 0
    // 0x1000204d 74 2e     je      0x1000207d
    // 0x1000204f 0f b7 1f  movzx   ebx, word ptr [edi]
    // 0x10002052 8b 7f 04  mov     edi, dword ptr [edi + 4]
    $op2 = { 74 36 8b 7f 08 83 ff 00 74 2e 0f b7 1f 8b 7f 04 }
    // 0x100020cf 74 70     je      0x10002141
    // 0x100020d1 81 78 05 8d 54 24 04      cmp     dword ptr [eax + 5], 0x424548d
    // 0x100020d8 75 1b     jne     0x100020f5
    // 0x100020da 81 78 08 04 cd ?? ??      cmp     dword ptr [eax + 8], 0xc22ecd04
    $op3 = { 74 70 81 78 05 8d 54 24 04 75 1b 81 78 08 04 cd }

condition:
    all of them
```

IOCs

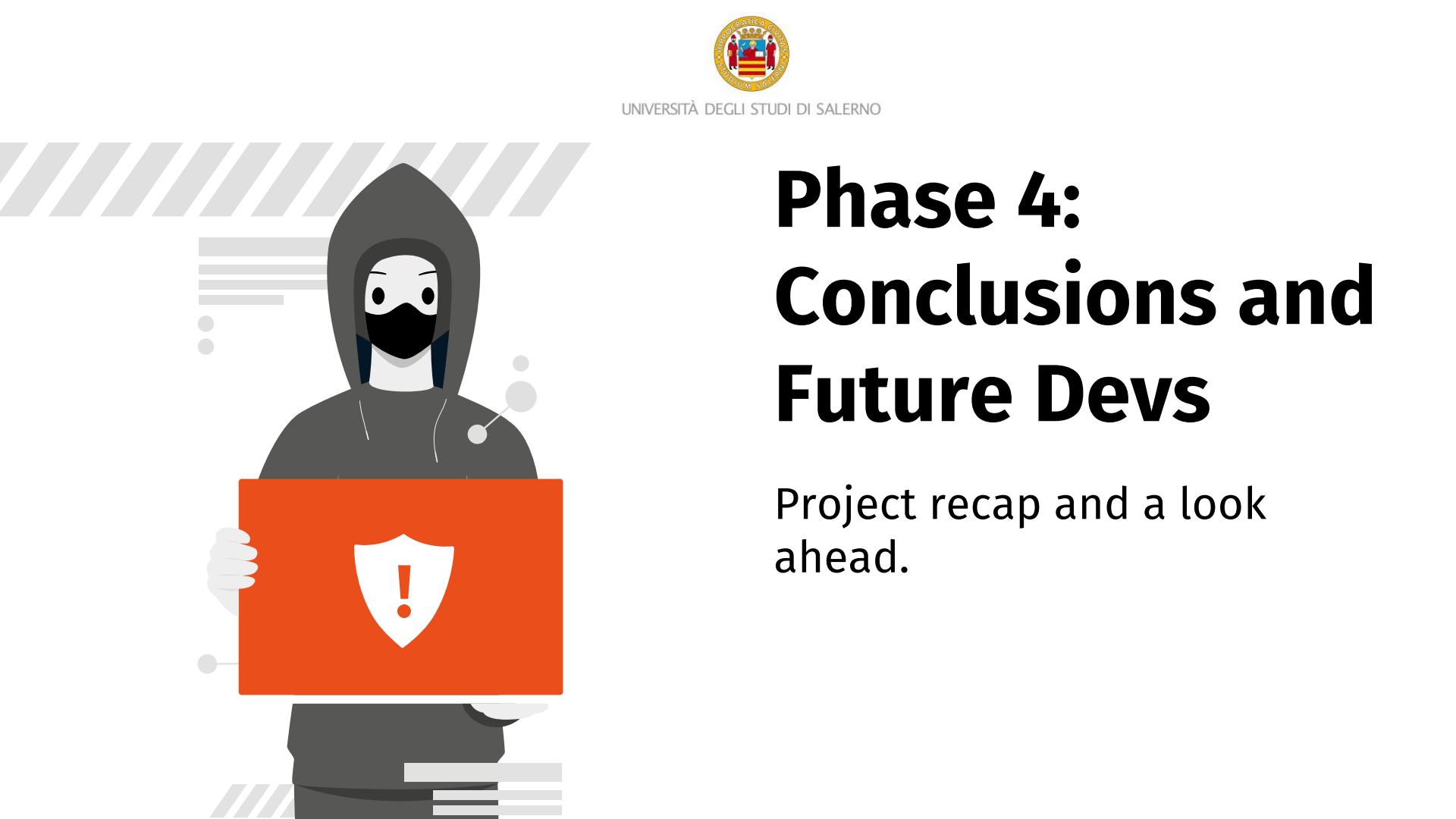
(Indicators Of Compromison)



The final output of the String and Signature Extractor is a structured list of **Indicators of Compromise (IOCs)**.

- **Embedded strings and signatures:** match known malware behaviors via YARA rules.
- **File hashes (MD5, SHA1, SHA256):** uniquely identify each malware sample.
- **IP addresses and URLs:** reveal possible command-and-control servers or download sources.
- **Registry keys and file paths:** show persistence or system modification attempts.

All extracted IOCs are then stored and passed to the **Reporting Engine** and **Behavioral Predictor** for correlation and threat classification.



Phase 4: Conclusions and Future Devs

Project recap and a look ahead.

Project Recap

1

Module 1

Malware Collector implemented with the most popular APIs, in a secure environment, to be completed with advanced malware capture systems.

2

Module 2

Disassembler Module with Ghidra (headless mode) and radare2, full use of basic tools.

3

Module 3

String and Signature Extractor with weighted scoring system and use of YARA Rules Datasets.

Development Roadmap

Module	Design	Development	Complete
Malware Sample Collector	✓	✓	✗
Disassembler Module	✓	✓	✓
String and Signature Extractor	✓	✓	✓
Behavioral Predictor	✓	✗	✗
Reporting Engine	✓	✗	✗

Future Devs

Behavioral Predictor

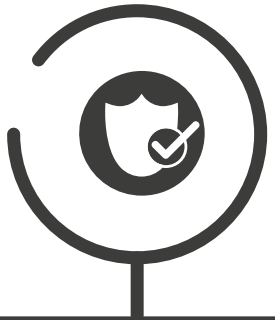
Sviluppo con modelli di apprendimento automatico avanzati o LLM di un sistema di predizione AI based.

Report Engine

Sviluppo di un Report Engine per avere un quado generale dell'analis semplice, intuitivo e ripetibile.

Dynamic Analysis

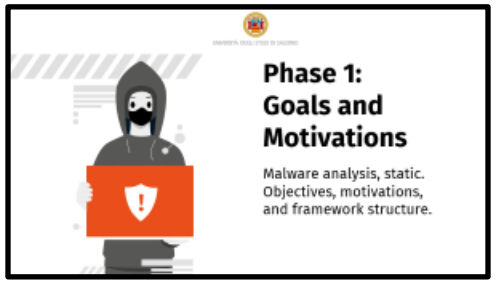
Integrazione con la Dynamic Analysis, sviluppo di una UI intuitiva per test dinamici, parallelizzazione dei processi statici e dinamici.

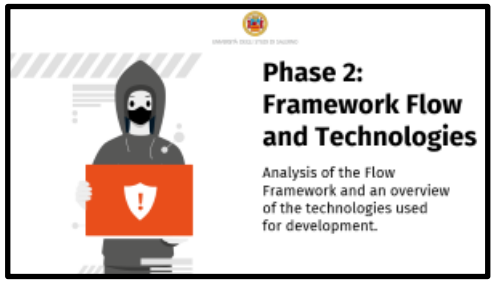


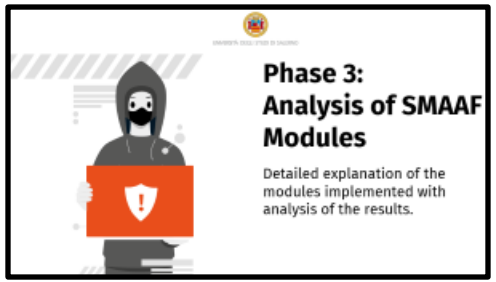
Threat Intelligence

Aggiunta di sistemi di condivisione STIX/TAXII per integrare i report con piattaforme di collaborazione tra enti e organizzazioni.

End of presentation







Antonio Garofalo
GitHub

Thank you for your attention.